

C# でマルチテナントと動的 OIDC 認証を実現する技術構成

近畿大学 マイコン部 竹内一希

.NET Aspire で開発は“もっときれいに”なる

Aspire が解決する開発の「面倒」なこと

分散アプリケーション開発には、多くの「面倒」がつきまといます。Aspire はそれらを解決します。

- “ローカルで DB も動かしたいけど、Docker のセットアップが面倒…” → **.AppHost が解決**
- “サービス A からサービス B の URL はどうやって知るの？設定に書きたくない…” → **サービスディスカバリが解決**
- “全サービスのログを一箇所で見たい！問題追跡がづらい…” → **Aspire ダッシュボードが解決**
- “一時的なネットワークエラーで処理が失敗して困る…” → **回復性 (Polly) が解決**

課題解決①: 複雑なローカル環境の簡素化

Aspire の解決策: .AppHost プロジェクトに、C# コードでインフラ構成を記述できます。

```
1 // src: TreeTopic.AppHost/AppHost.cs C#
2 var builder =
3   DistributedApplication.CreateBuilder(args)
4
5   var postgres =
6     builder.AddPostgres("postgres")
7       .WithPgAdmin();
8
9     var tenantDb =
10      postgres.AddDatabase("treetopic-
11        tenants");
12
13      var appDb =
14        postgres.AddDatabase("SharedApp");
15
16
```

```
10 var projectBuilder =
11     builder.AddProject<TreeTopic>("treetopic")
12       .WithReference(tenantDb)
13       .WithReference(appDb)
14       .WaitFor(postgres);
15
16 if (builder.Environment.IsDevelopment())
17 {
18     var keycloak =
19       builder.AddKeycloak("keycloak", ...);
20
21     projectBuilder.WithReference(keycloak)
22 }
23
24 builder.Build().Run();
```

課題解決①: 複雑なローカル環境の簡素化

主なポイント:

- `AddPostgres(...)`: PostgreSQL コンテナを定義。
- `.WithPgAdmin()`: DB 管理ツール(PgAdmin)を自動で追加。
- `AddDatabase(...)`: コンテナ内に DB を作成。
- `AddProject<...>(...)`: Web API プロジェクトを追加。
- `.WithReference(...)`: プロジェクトに DB 等の接続情報を自動で連携。
- `.WaitFor(...)`: DB 等の準備が完了するまでプロジェクトの起動を待機。
- `AddKeycloak(...)`: 開発用に Keycloak (認証サーバー) コンテナを追加。
- `builder.Build().Run()`: Aspire アプリケーションをビルドして起動。

課題解決① (補足): 任意のコンテナを実行

.AddPostgres()のような定義済みのメソッドだけでなく、AddContainer()を使えば、どんな Docker コンテナでも Aspire アプリケーションの一部として管理できます。

例: 開発用のメールサーバー(Mailpit)を追加

```
1 // Docker Hubの任意のイメージを指定
2 var mailpit = builder.AddContainer("mailpit", "axllent/mailpit")
3                             .WithHttpEndpoint(targetPort: 8025, hostPort: 1025);
```

C#

- AddContainer("name", "image:tag"): 好きな Docker イメージでコンテナを定義します。
- .WithHttpEndpoint(...): コンテナのポートをホストに公開し、サービスディスカバリの対象にします。
- .WithReference(mailpit): treetopic プロジェクトに mailpit の URL(http://localhost:1025 など)が環境変数として自動で連携されます。

課題解決① (補足): Next.js(Node.js)フロントエンドの実行

Aspire では、Node.js ベースのフロントエンドもバックエンドと同様に管理できます。

開発時 (npm / ホットリロード)

```
1 // `npm run dev` を実行し、ホットリロードを有効化
2 var frontend = builder.AddNpmApp("frontend", "../path/to/next-app")
3     .WithHttpEndpoint(port: 3000);
```

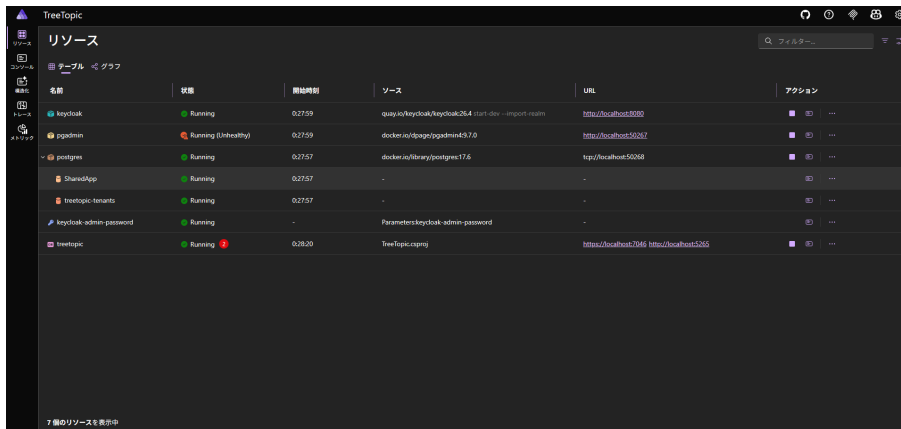
C#

本番環境 (Docker)

```
1 // ビルド済みのDockerイメージを実行
2 var frontend = builder.AddContainer("frontend", "my-next-app:latest")
3     .WithHttpEndpoint(port: 3000);
```

C#

課題解決②: サービス間の「見えない」問題を可視化



The screenshot shows the TreeTopic application interface. On the left is a sidebar with navigation icons for Resources, Dashboard, Traces, and Metrics. The main area is titled 'リソース' (Resources) and contains a table with the following data:

名前	状態	開始時刻	ソース	URL	アクション
keycloak	Running	0:27:59	quay.io/keycloak/keycloak:26.4 start-dev --import-realm	http://localhost:8080	[Stop] [Refresh] [More]
pgadmin	Running (Unhealthy)	0:27:59	docker.io/dpage/pgadmin4:9.7.0	http://localhost:50267	[Stop] [Refresh] [More]
postgres	Running	0:27:57	docker.io/library/postgres:17.6	tcp://localhost:50268	[Stop] [Refresh] [More]
SharedApp	Running	0:27:57	-	-	[Stop] [Refresh] [More]
tree-topic-tenants	Running	0:27:57	-	-	[Stop] [Refresh] [More]
keycloak-admin-password	Running	-	Parameters: keycloak-admin-password	-	[Stop] [Refresh] [More]
tree-topic	Running [Error]	0:28:20	TreeTopic.cpmj	https://localhost:7046 https://localhost:5265	[Stop] [Refresh] [More]

At the bottom of the table, it says '7 個のリソースを表示中' (Showing 7 resources).

課題: マイクロサービス間で問題が起きた時、どのサービスのログを見ればいいのか分からない。リクエストの追跡が困難。

Aspire の解決策: Aspire ダッシュボード

課題解決②: サービス間の「見えない」問題を可視化

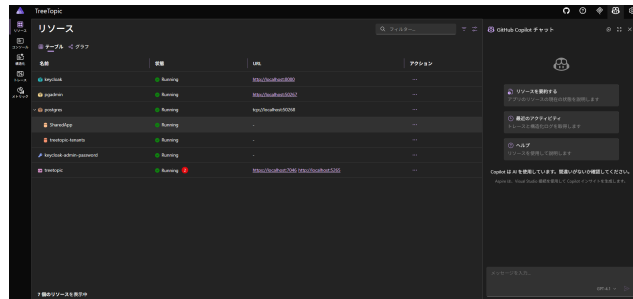
- **構造化ログ:** 全サービスのログをまとめて確認・検索。
- **分散トレース:** リクエストがどのサービスを経由したかを可視化し、ボトルネックやエラー箇所を特定。
- **メトリクス:** 各サービスの CPU・メモリ使用量を監視。

Copilot in Aspire Dashboard

.NET Aspire のダッシュボードには、GitHub Copilot が統合されています。これにより、AI デバッグアシスタントとして、開発者はより迅速に問題解決を行えます。

ダッシュボードでの Copilot 活用例:

- 大量のログメッセージを要約
- 複数サービスにまたがるエラーの根本原因を調査
- 分散トレースからパフォーマンスの問題を特定
- 不明なエラーコードの意味を解説



課題解決③: 全サービスで「品質」を標準化

課題: サービスごとにログ形式が違ったり、回復性ポリシー（リトライなど）を実装し忘れたりする。

Aspire の解決策: .ServiceDefaults プロジェクト

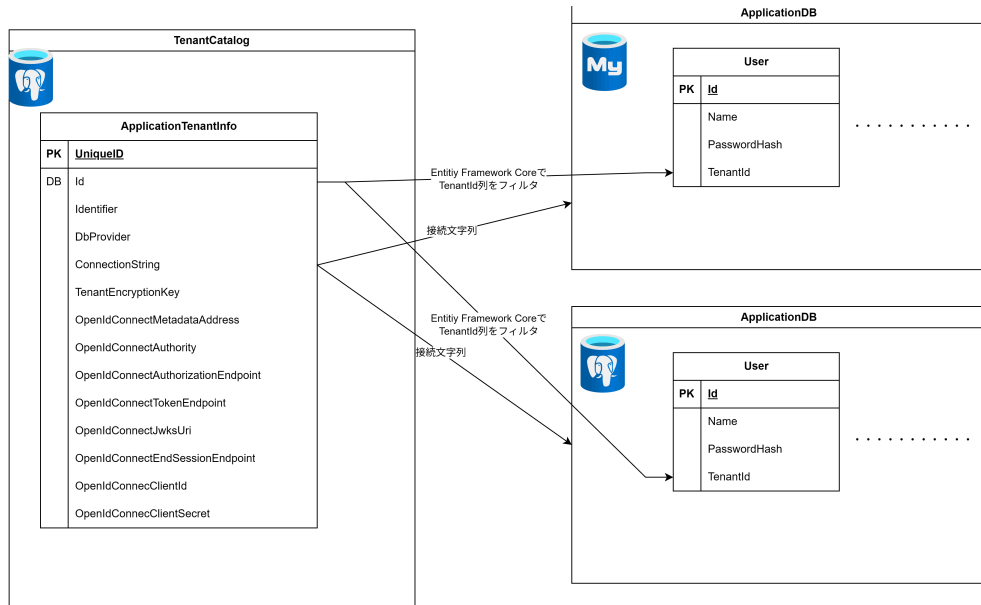
- **OpenTelemetry:** 統一形式のログ・トレース・メトリクスを自動収集。
- **標準の回復性:** HttpClient にリトライ処理などを自動追加。
- **ヘルスチェック:** 標準化された方法で稼働状況を報告。

各サービスは `builder.AddServiceDefaults()` の一行を追加するだけで、これらのベストプラクティスを導入できます。(

```
src: raw(text: "TreeTopic/Program.cs", block: false),  
)
```

Finbuckle.MultiTenant: マルチテナント実装

テナント解決の仕組み（リクエストの流れ）



テナント解決の仕組み（リクエストの流れ）

1. **リクエスト到着**: 認証済みユーザーからのリクエストが到着します。
2. **ミドルウェア実行**: `Finbuckle.MultiTenant` のミドルウェアが実行されます。
3. **ClaimStrategy 起動**: `CustomClaimStrategy (src: Services/CustomClaimStrategy.cs)` が呼び出され、ユーザーの JWT から tenant クレーム（例: “tenant-a”）を読み取ります。
4. **Store 起動**: `EFCoreMultiTenantStore (src: Services/EFCoreMultiTenantStore.cs)` が、テナントカタログ DB に “tenant-a” を問い合わせ、対応するテナント構成情報を取得します。
5. **テナント情報設定**: 取得されたテナント情報 (`ApplicationTenantInfo`) が、このリクエストのコンテキストに設定されます。
6. **DB 接続切り替え**: `ApplicationDbContext` が使用される際、このテナント情報に基づいた正しいデータベース接続文字列が透過的に使用されます。

詳細: データ分離の実現

テナントごとのデータ分離は、DbContext が使われる際に、動的に接続文字列を切り替えることで実現されます。

```
1 // src: TreeTopic/Program.cs
2 builder.Services.AddDbContext<ApplicationDbContext>((sp, options) =>
3 {
4     options.UseMultiTenantDatabase(sp);
5 });
```

C#

ApplicationDbContext を DI コンテナに登録する際、ファクトリ関数を指定します。この中で自作の拡張メソッド UseMultiTenantDatabase() を呼び出します。

このメソッドが内部で IMultiTenantContextAccessor を使い、現在のテナント情報から接続文字列を取得して、透過的に DbContext に設定します。これにより、同じ ApplicationDbContext のコードを使いながらも、リクエストのテナントに応じて接続先の DB が自動で切り替わります。

動的 OIDC 認証

テナントごとの動的な OIDC 認証

このプロジェクト最大の特徴は、テナントごとに異なる OIDC プロバイダ（認証サーバー）を動的に切り替えられる点です。

- **なぜ必要か？**: 顧客（テナント）が自社で使用している既存の認証基盤（例: 自社運用の Keycloak, Azure AD）でログインしたい、というエンタープライズ向けの要求に応えるため。
- **実現方法**: ASP.NET Core の OIDC 認証イベントをフックし、リクエストの都度、テナントに応じた OIDC 設定を動的に適用します。 `Extensions/OpenIdConnectExtensions.cs`

動的 OIDC 認証の仕組み（イベントの流れ）

1. **ログイン開始:** ユーザーがテナント “tenant-a” を指定してログインを開始します。（例: /auth/login?tenant=tenant-a）
2. **OIDC イベント発生:** OIDC の OnRedirectToIdentityProvider イベントが発生します。
3. **設定取得:** イベントハンドラ内で、“tenant-a” の OIDC 構成（Keycloak のエンドポイント URL や ClientId 等）をテナントカタログ DB から取得し、OIDC プロバイダの Well-known エンドポイントから自動で設定情報を取得します。
4. **OIDC オプション動的設定:** 取得した情報で、このリクエストの OIDC オプションを動的に上書きします。
5. **リダイレクト:** ユーザーは “tenant-a” 専用の Keycloak 認証画面にリダイレクトされます。
6. **認証成功後イベント:** 認証後、OnAuthorizationCodeReceived イベントが発生します。
7. **ClientSecret 取得・トークン要求:** ここで再度テナントを特定し、対応する ClientSecret を DB から（復号して）取得し、アクセストークンを要求します。これにより、テナントごとの秘密情報を安全に利用できます。

アーキテクチャの相乗効果

動的 OIDC 認証の仕組み（イベントの流れ）

これら 3 つの技術は、以下のように連携して動作します。

1. **開発時:** 開発者が **.NET Aspire** でプロジェクトを起動すると、API サーバー、DB、Keycloak が自動で立ち上がります。
2. **認証時:** ユーザーがログインしようとする、動的 **OIDC** の仕組みが働き、リクエストに応じたテナントの認証サーバーへリダイレクトされます。
3. **認証後:** 認証が成功すると、ユーザーのクレームに **tenant** 識別子が追加されます。
4. **API アクセス時:** 以降の API リクエストでは、**Finbuckle.MultiTenant** の **CustomClaimStrategy** が **tenant** クレームを読み取り、現在のテナントを確定させ、DB アクセス時に適切なテナントデータベースへ接続します。

この連携により、柔軟性と拡張性、そしてセキュリティを高いレベルで両立した、モダンなマルチテナントアプリケーションが実現されています。